Evolving the Computational Notebook: A Two-Dimensional Canvas for Enhanced Human-AI Interaction

Konstantin Grotov
Neapolis University Paphos
Paphos, Cyprus
konstantin.grotov@gmail.com

Dmitry Botov
Neapolis University Paphos
Paphos, Cyprus
AI Future Hub
dmbotov@gmail.com

***Abstract*—Computational notebooks, while essential for data science, are limited by their one-dimensional interface, which poorly aligns with non-linear developer workflows and complicates collaboration and human-AI interaction. In this work, we focus on features of Computational Canvas, a novel two-dimensional interface that evolves notebooks to enhance data analysis and AI-assisted development within integrated development environments (IDEs). We present vital features, including freely arrangeable code cells, separate environments, and improved output management. These features are designed to facilitate intuitive organization, visual exploration, and natural collaboration with other users and AI agents. We also show the implementation of Computational Canvas with designed features as a Visual Studio Code plugin. By shifting from linear to two-dimensional spatial interfaces, we aim to significantly boost developers' productivity in data exploration, experimentation, and AI-assisted development, addressing the current limitations of traditional notebooks and fostering more flexible, collaborative data science workflows.**

***Index Terms*—Computational Notebooks, HAX, AI Agents**

I. Introduction

Computational notebooks are the favored medium for writing code and follow the paradigm of literal programming [1]. Nowadays, computational notebooks have played a crucial role in different areas, like data analysis [2], science [3], [4], and education [5], [6].

The framework Jupyter [7], renowned for its computational notebooks and extensively utilized by data scientists [2], is distinguished by its ability to develop code in a stateful manner. This means that each new code execution incrementally updates the notebook's runtime state, allowing users to iterate more swiftly without re-running all the previous code.

Although computational notebooks are a convenient medium for data analysis, the code users write in them is often of worse quality and more entangled structure rather than the one they write in scripts [8]. This entanglement saves beyond the single cell level, as the notebook cell order often doesn't correlate with the actual highly non-linear developer's workflow [9], [10]. Moreover, notebooks struggle with very poor reproducibility [11], [12], especially in domains related to science [13], where reproducibility is crucial. The problems mentioned above make notebooks a medium that is not reliable for end-to-end software engineering but is the perfect choice for retrieving insights from data and experimentation [10], [14].

With the rise of artificial intelligence and foundational models [15], [16], the literate programming paradigm has been changing accordingly [17], making the initial notebook interface limited in providing a comprehensible human-AI interaction. AI assistants often generate code snippets, modify multiple interdependent cells, or suggest alternative approaches that require users to revisit and restructure existing code. In a linear notebook, these interactions can quickly become chaotic, leading to confusion about the execution order, unintended overwrites, and cognitive overload for users trying to keep track of changes [18].

Recent studies proposed developing notebooks to streamline the experimentation workflow and collaboration rather than transferring and mimicking the existing tools from integrated development environments (IDEs) to notebooks [19]–[21]. Moreover, the studies on changing the one-dimensional structure of the notebooks to a two-dimensional one showed a boost in developer productivity [22]–[24]. Although the mentioned studies highlighted user interest in two-dimensional notebooks, their implementation was limited to multicolumn layouts, so they did not fully explore the new functionalities that true two-dimensional notebooks could offer.

To fill this gap, in this work, we propose a specific set of features that could facilitate the developer's experimentation in Computational Canvas[1] – the new interface enhances the traditional computational notebook by providing a two-dimensional workspace. Our approach allows users to create multiple, safely separate environments that branch off from the main runtime state, making it easier to experiment within a unified space. In this work, we outline the key features of this interface and explain how it can facilitate human-AI interaction. Moreover, we have implemented the computational canvas with the designed features as a plugin for Visual Studio Code. This approach simplifies the system, fostering non-linear experimentation and seamless collaboration without

[1] https://github.com/konstantgr/computational-canvas

2025 IEEE/ACM Second IDE Workshop (IDE) | 979-8-3315-0188-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/IDE66625.2025.00009

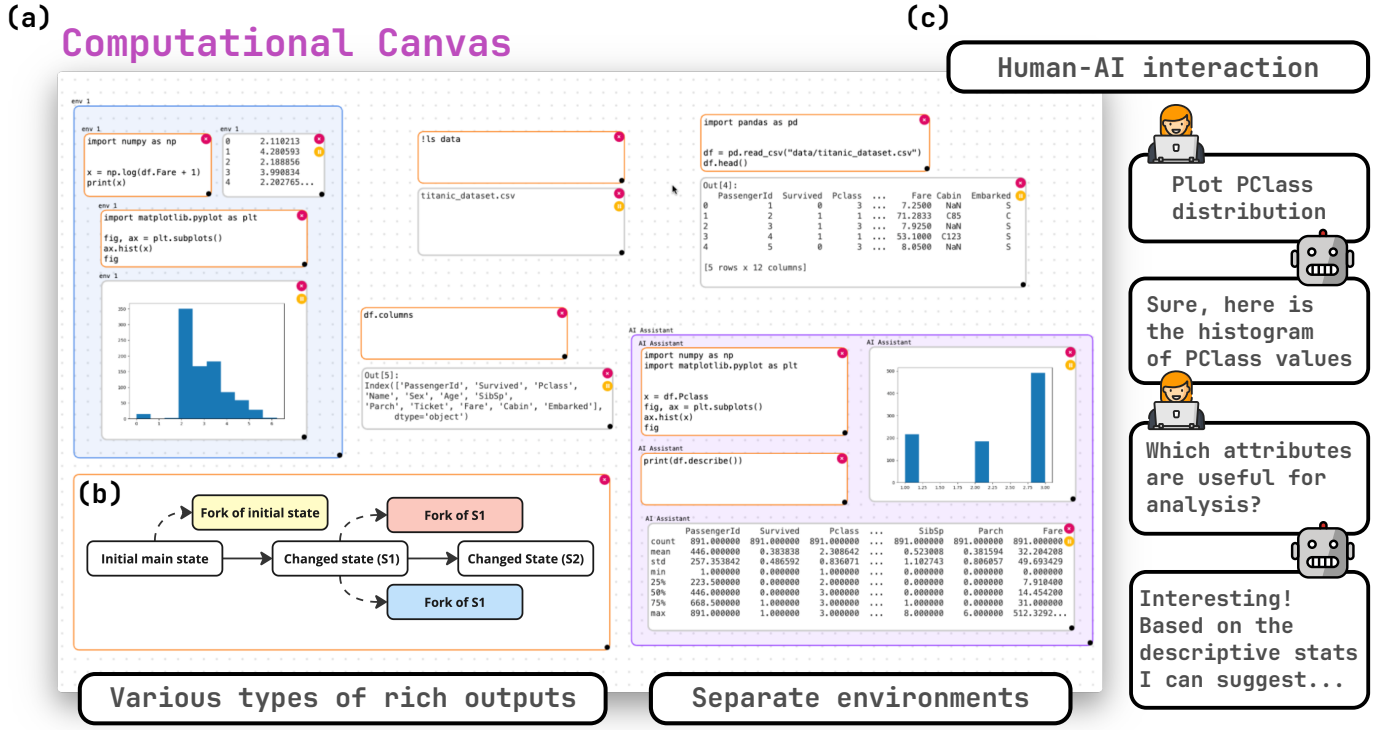

Fig. 1. **(a)** Prototype interface of the computational canvas. Orange cells in the canvas are the executable cells, the grey cells are the outputs, and the colorful areas are the separate environments with independent runtime states. **(b)** Schematic representation of the forking mechanism while creating a new environment. **(c)** A schematic representation of a chat between a user and an AI assistant is in Computational Canvas. The agent creates the dedicated environment that executes the user request.

conflicts right in the IDE. These collaborations can occur not only between developers but also with AI assistants.

II. Computational Canvas

The Computational Canvas is a platform that allows users to experiment with code and analyze data in a two-dimensional plane. The canvas prototype is elucidated in Figure 1(a). The canvas's core features include a flexible arrangement of executable cells, support for different rich output formats, and the creation of a separate environment with forks of the main runtime state. This section will describe these features in more detail.

A. Cells Arrangement

Development in computational canvas is a process of creating executable cells with the code and obtaining the execution output, similar to workflow in the Jupyter Notebook. However, contrary to the vanilla Jupyter Notebook, the cells in the computational canvas are not bound to any grid. Any cell could be moved to the desired position of the developer, making it possible to organize the cells in groups more naturally than in a linear notebook. The interface was inspired by the Miro board[2] interface, which is widely used for collaborative brainstorming.

[2]https://miro.com

It's worth mentioning that the computational canvas is designed to be infinite, so the user can separate their workflow and comfortably utilize the canvas by dividing it into the needed groups. In contrast with the common one-dimensional computational notebook, the interface does not implicitly bind the user to follow the linear workflow, thus facilitating non-linear development.

B. Rich Outputs Manipulation

After the cell is executed, an additional output cell is created below it. Like the standard notebook, the computational canvas supports textual and graphical outputs, making it possible to use it for data analysis, where visual representation is often crucial. Whenever the same code cell is executed again, the output cell will update accordingly, even if it is located in different positions on the canvas. This feature allows users to separate the canvas into the area of executable code cells and the area where the users expect to review the output, making the iterations with the code or debugging more comprehensive.

Moreover, compared to the vanilla Jupyter Notebook, we're proposing the feature of detaching the output from its parent cell and fixing its state at the moment. On the one hand, it could help users develop and keep all gained insights in one place without worrying about occasionally rewriting them. Additionally, it provides a more structured way to organize

outputs, which can be beneficial for presentations. Figure 2 illustrates an example of detached outputs.

C. Separate Environments

The developer's workflow in computational notebooks is highly non-linear. By leveraging a two-dimensional space, users can break free from linear constraints. However, this brings another issue: ensuring that the code remains aligned with its intended purpose during the workflow.

For example, while working with a Pandas table `df`, numerous transformations and manipulations might be applied throughout the development process. As a result, code that was written and executed at the start may become obsolete or lead to unexpected results as the table evolves. Even worse, during the experimentation, crucial data could be corrupted, and the whole reproduction of the pipeline would be needed.

To address this issue, we propose a feature of separate environments within the computational canvas. This approach will enable developers to experiment freely within these environments while ensuring their main pipeline remains well-organized and traceable.

Initially, the first cell execution in the canvas creates a main runtime – a process that stores the variables and functions declared in the previous executions. At any time, the user can create a new distinct environment, which will initially be forked from the main runtime but will not be referenced anymore. Then, this environment could be used for safe exploration and experimentation. Figure 1(b) shows the schematic forking diagram.

The user interface displays the environment as a colored, interactive area that can be moved around the canvas or resized. When placed in a particular environment, a new cell will be executed using its associated runtime. Since each environment is a distinct instance, it can be moved along with all the cells it contains. Figure 1(a) illustrates this with two environments, represented by purple and blue areas.

D. Collaborative Exploration

The computational canvas could be a promising tool for more natural and collaborative exploration than the computational notebooks. By leveraging the power of separate environments, users can create distinct partitions for different collaborators, control runtime permissions, and establish shared spaces within a single canvas.

It's worth mentioning that the user experience of collaboration within a computational canvas primarily emphasizes brainstorming and gathering insights, such as generating images, tables, and other outputs. Rather than focusing on co-developing and producing reproducible, reliable code, the platform facilitates the exchange of ideas and knowledge among collaborators.

III. AI Assistance in Computational Canvas

AI assistants (AI agents) based on foundation models [25] have demonstrated impressive coding and data analysis abilities. With coding proficiency and advanced reasoning skills, cooperative data exploration and analysis with AI agents could be performed even using natural language only. It allows users to focus more on value than the exact implementation. However, integrating them seamlessly into a user's workflow within notebooks presents a significant challenge in terms of user experience [26]. For example, while debugging in a notebook with agents, users often could experience a cognitive overload due to its fast and autonomous interaction with the notebook [27]. Thus, it is necessary to ensure that the agent assists users rather than fully controls their workflow.

Various types of AI agents could be incorporated into the computational canvas, enhancing user productivity while keeping the user in control. These could include **data analysis assistants** that provide insights and visualizations, **documentation helpers** that automatically generate explanations for code cells or organize the outputs, **debugging agents** that analyze errors in the secure environment and suggest fixes, and **collaborative brainstorming agents** that could offer creative ideas during experimentation. Figure 1(c) shows the example of user interaction with the agent, which could address the requests by creating a separate environment, executing code autonomously, and presenting the results to the user.

Computational canvases provide an interactive platform for cooperative experimentation with autonomous collaborators. By offering a secure, dedicated environment, users can safely grant permissions for code execution and debugging while also facilitating collaborative brainstorming sessions. The two-dimensional nature of the canvas allows for the indication and grouping of cells used by an agent for a particular request in a natural, comprehensible manner.

IV. Canvas Integration in IDE

The computational canvas could be a great assistant for developers in IDEs. We've implemented Computational Canvas as a plugin for Visual Studio Code[3]. To create a canvas within the IDE, users simply need to create a new file with the extension `.2dntb`. If the file is initially empty, it will be automatically filled with the necessary content upon the first execution, making it ready for use as a canvas file. Additionally, we have ensured that canvas files are compatible with standard Jupyter Notebook `.ipynb` files. The user can either load the existing notebook into the canvas or save the canvas as a Jupyter notebook with the same cell order as they were created in the canvas. The example of the canvas user interface in the VS Code is shown in Figure 2.

Much like Project Jupyter, a dedicated server runs whenever a cell is executed within the canvas. The server can manage multiple canvases simultaneously. When a user executes a cell, the client sends a POST request to the server, which then processes the code using a Python REPL environment. When a cell is executed in a newly created environment, the server forks a new Python REPL session from the main one.

Furthermore, beyond the intuitive user interface, we have introduced the ability to interact with the canvas through an

[3]https://code.visualstudio.com/

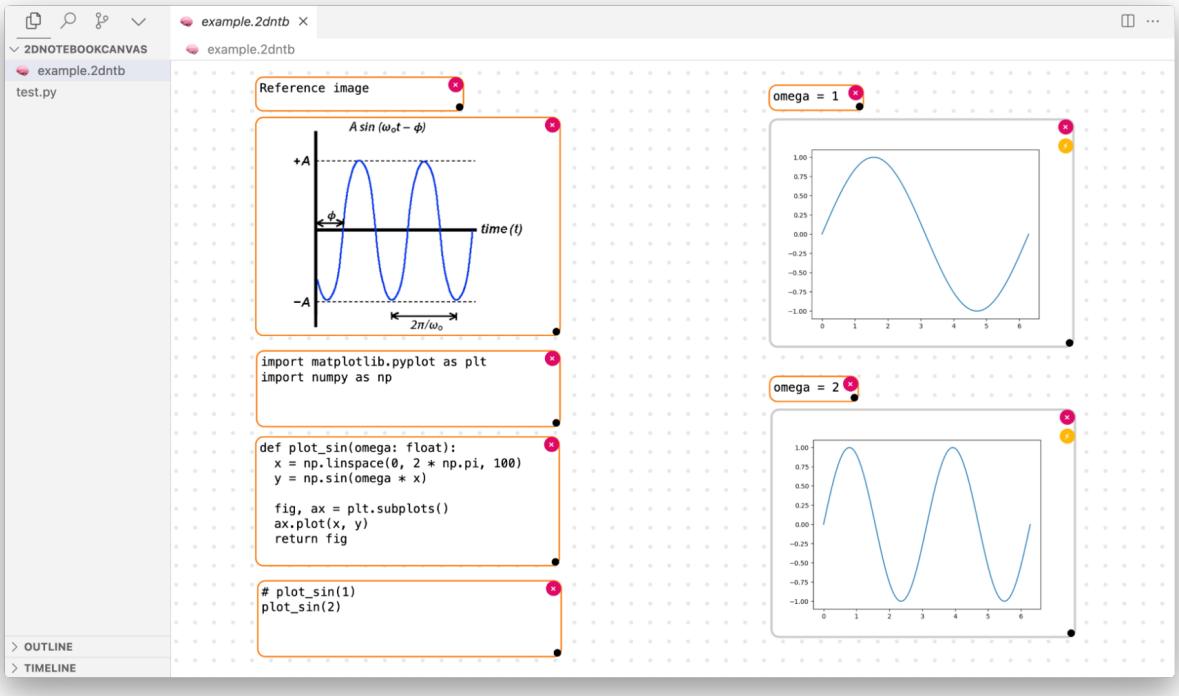

Fig. 2. example of detached outputs in the computational canvas opened in Visual Studio Code. The left side showed the code, which was executed with different parameters. The right side showed the outputs, which were detached and grouped together.

API. This advancement allows AI Agents to interface with the canvas, keeping the agent's logic separate from the canvas. This also could allow other developers to add their own agents easily.

V. Conclusion

The evolution of computational notebooks has often led to being overwhelmed for users in the new era of literate programming. The Computational Canvases, a two-dimensional platform, is a promising candidate for the better in-IDE experience of intensive experimentation and data analysis. In this work, we outlined the key features of Computational Canvases, demonstrating how they balance simplicity with functionality, supporting exploration, iterative experimentation, collaborative work, and seamless human-AI interaction in IDE.

By transitioning from linear notebooks to unordered, two-dimensional interfaces, Computational Canvases empower users to focus more on gaining insights and less on working with the code in the common script manner.

We believe Computational Canvases can become an indispensable tool for developers, data scientists, and researchers. Future research could explore the human experience while working on canvases and the patterns while experimenting with the canvas. Ultimately, computational canvases open new frontiers in interacting with code, data, and ideas, opening the way for more creative development experiences.

References

[1] Donald Ervin Knuth. Literate programming. *The computer journal*, 27(2):97–111, 1984.

[2] Jeffrey M Perkel. Why jupyter is data scientists' computational notebook of choice. *Nature*, 563(7732):145–147, 2018.

[3] Marijan Beg, Juliette Taka, Thomas Kluyver, Alexander Konovalov, Min Ragan-Kelley, Nicolas M Thiéry, and Hans Fangohr. Using jupyter for reproducible scientific workflows. *Computing in Science & Engineering*, 23(2):36–46, 2021.

[4] Bernadette M Randles, Irene V Pasquetto, Milena S Golshan, and Christine L Borgman. Using the jupyter notebook as a tool for open science: An empirical study. In *2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*, pages 1–2. IEEE, 2017.

[5] Jeremiah W Johnson. Benefits and pitfalls of jupyter notebooks in the classroom. In *Proceedings of the 21st annual conference on information technology education*, pages 32–37, 2020.

[6] Lorena A Barba, Lecia J Barker, Douglas S Blank, Jed Brown, Allen B Downey, Timothy George, Lindsey J Heagy, Kyle T Mandli, Jason K Moore, David Lippert, et al. Teaching and learning with jupyter. *Recuperado: https://jupyter4edu. github. io/jupyter-edu-book*, pages 1–77, 2019.

[7] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks – a publishing format for reproducible computational workflows. In F. Loizides and B. Schmidt, editors, *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87 – 90. IOS Press, 2016.

[8] Konstantin Grotov, Sergey Titov, Vladimir Sotnikov, Yaroslav Golubev, and Timofey Bryksin. A large-scale comparison of python code in jupyter notebooks and scripts. In *Proceedings of the 19th International Conference on Mining Software Repositories*, pages 353–364, 2022.

[9] Souti Chattopadhyay, Ishita Prasad, Austin Z Henley, Anita Sarma, and Titus Barik. What's wrong with computational notebooks? pain points, needs, and design opportunities. In *Proceedings of the 2020*

CHI conference on human factors in computing systems, pages 1–12, 2020.

[10] Dhivyabharathi Ramasamy, Cristina Sarasua, Alberto Bacchelli, and Abraham Bernstein. Workflow analysis of data science code in public github repositories. *Empirical Software Engineering*, 28(1):7, 2023.

[11] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. A large-scale study about quality and reproducibility of jupyter notebooks. In *2019 IEEE/ACM 16th international conference on mining software repositories (MSR)*, pages 507–517. IEEE, 2019.

[12] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. Understanding and improving the quality and reproducibility of jupyter notebooks. *Empirical Software Engineering*, 26(4):65, 2021.

[13] Sheeba Samuel and Daniel Mietchen. Computational reproducibility of jupyter notebooks from biomedical publications. *GigaScience*, 13:giad113, 2024.

[14] Adam Rule, Aurélien Tabard, and James D Hollan. Exploration and explanation in computational notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–12, 2018.

[15] OpenAI. Gpt-4 technical report. *ArXiv*, abs/2303.08774, 2023.

[16] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[17] Kensen Shi, Deniz Altınbüken, Saswat Anand, Mihai Christodorescu, Katja Grünwedel, Alexa Koenings, Sai Naidu, Anurag Pathak, Marc Rasi, Fredde Ribeiro, et al. Natural language outlines for code: Literate programming in the llm era. *arXiv preprint arXiv:2408.04820*, 2024.

[18] Andrew M McNutt, Chenglong Wang, Robert A Deline, and Steven M Drucker. On the design of ai-powered code assistants for notebooks. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2023.

[19] Sergey Titov, Konstantin Grotov, and Ashwin Prasad S. Venkatesh. Hidden gems in the rough: Computational notebooks as an uncharted oasis for ides. In *Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments*, pages 107–109, 2024.

[20] April Wang, Zihan Wu, Christopher Brooks, and Steve Oney. Don't step on my toes: resolving editing conflicts in real-time collaboration in computational notebooks. In *Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments*, pages 47–52, 2024.

[21] Chengbo Zheng, Dakuo Wang, April Yi Wang, and Xiaojuan Ma. Telling stories from computational notebooks: Ai-assisted presentation slides creation for presenting data science work. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pages 1–20, 2022.

[22] Elizabeth Christman. *2D Jupyter: Design and Evaluation of 2D Computational Notebooks*. PhD thesis, Virginia Tech, 2023.

[23] Jesse Harden, Elizabeth Christman, Nurit Kirshenbaum, Mahdi Belcaid, Jason Leigh, and Chris North. "there is no reason anybody should be using 1d anymore": Design and evaluation of 2d jupyter notebooks. *Graphics Interface 2023*, 2023.

[24] Jesse Harden, Elizabeth Christman, Nurit Kirshenbaum, John Wenskovitch, Jason Leigh, and Chris North. Exploring organization of computational notebook cells in 2d space. In *2022 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 1–6. IEEE, 2022.

[25] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O'Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, et al. Practices for governing agentic ai systems. Technical report, OpenAI Technical Report. https://cdn. openai. com/papers/practices-for . . . , 2023.

[26] Thomas Weber and Sven Mayer. From computational to conversational notebooks. *arXiv preprint arXiv:2406.10636*, 2024.

[27] Konstantin Grotov, Artem Borzilov, Maksim Krivobok, Timofey Bryksin, and Yaroslav Zharov. Debug smarter, not harder: Ai agents for error resolution in computational notebooks. *arXiv preprint arXiv:2410.14393*, 2024.